A PROJECT REPORT ON

Abstractive Summarization of Research Papers using Advanced NLP Techniques

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Ajit Abhyankar	Exam No: B400050312
Aniket Rajesh	Exam No: B400050319
Vedant Kokane	Exam No: B400050451
Vivek Gotecha	Exam No: B400050632

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

Abstractive Summarization of Research Papers of Research Papers using Advanced NLP Techniq

Submitted by

Ajit Abhyankar	Exam No: B400050312
Aniket Rajesh	Exam No: B400050319
Vedant Kokane	Exam No: B400050451
Vivek Gotecha	Exam No: B400050632

are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. B. A. Sonkamble** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. B. A. Sonkamble
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Abstractive Summarization of Research Papers'****.*

We would like to take this opportunity to thank ***Dr. B. A. Sonkamble*** *giving us all the help and guidance we needed. We are really grateful to him for his kind support. His valuable suggestions were very helpful.*

We are also grateful to ***Dr. G. V. Kale****, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to* ***Dr. A. R. Deshpande****, BE project coordinator, for her timely guidance and support.*

In the end our special thanks goes to our honorable ***Dr. P. T. Kulkarni*** *, Director, PICT for their continued support and valuable input during the ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Ajit Abhyankar
Aniket Rajesh
Vedant Kokane
Vivek Gotecha
(B.E. Computer Engg.)

ABSTRACT

Our project aims to develop a system that automatically summarizes NLP research papers using advanced AI techniques, specifically Transformer-based models and Pointer-Generator Networks. These models will perform abstractive text summarization, meaning the system will not just extract sentences from the research papers, but will generate concise, easy-to-read summaries that capture the key points in a natural language style. By training the model on the ScisummNet Scientific Dataset, which contains around 1000 NLP research papers, we will tailor it to produce accurate and relevant summaries for this specific domain. The inclusion of Pointer-Generator Networks ensures that the system can handle specialized terminology and technical language commonly found in research papers, balancing both abstraction and extraction for more precise summarization. This system will be particularly useful for researchers, students, and professionals, helping them save time while reviewing papers and improving the accessibility of complex information. Overall, it will enhance the way research papers are understood and reviewed, making the process more efficient and effective.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Overview

In recent years, the volume of academic literature has grown at an unprecedented rate across all scientific disciplines. Researchers, scholars, and students often find themselves overwhelmed by the sheer number of articles published each year, making it increasingly difficult to stay informed about the latest developments in their respective fields. This information overload presents a significant barrier to knowledge acquisition, effective literature reviews, and timely decision-making in research environments. Automatic text summarization has become a crucial area of study in natural language processing (NLP), aiming to tackle this challenge by providing condensed versions of long documents that preserve the most critical information. Summarization techniques can be broadly categorized into extractive and abstractive methods. While extractive summarization selects and compiles key sentences directly from the source text, abstractive summarization generates new phrases and sentences that may not appear in the original document, requiring deeper semantic understanding and language generation capabilities. In this project, we focus on abstractive summarization of research papers, leveraging two cutting-edge techniques: BART (Bidirectional and Auto-Regressive Transformers), a transformer-based encoder-decoder model known for its performance on generative NLP tasks, and the Pointer-Generator Network (PGN), which combines the flexibility of generation with the precision of copying directly from the source text. These models are applied to summarize the abstracts and introduction sections of scientific papers, aiming to produce fluent, coherent, and informative summaries that align closely with the original content's intent.

1.2 Motivation

The motivation for undertaking this project arises from the increasing difficulty faced by researchers in digesting the growing corpus of academic literature. Conducting a literature review or staying updated in a specific domain requires the ability to quickly comprehend the essence of multiple research articles. Reading full papers is time-consuming, and even abstracts can be lengthy or poorly written. Traditional extractive summarization tools, though fast, often produce summaries that lack context, coherence, or the ability to paraphrase and generalize information effectively. This situation creates a strong need for intelligent systems that can automatically generate concise, human-like summaries of research papers. Abstractive summarization offers a solution that mimics the way humans write summaries—by understanding the content and expressing it in a new, condensed form. This capability is particularly valuable in the academic domain, where understanding and synthesizing complex information are critical. The use of advanced models like BART and PGN in this project is motivated by their proven performance in various summarization and sequence-to-sequence tasks. BART, with its denoising autoencoder pretraining and transformer architecture, excels in generating fluent summaries. PGN, on the other hand, handles out-of-vocabulary words and repetition issues effectively by combining pointer and generator mechanisms. Together, these models provide a powerful toolkit for

tackling the abstractive summarization of research texts.

1.3 Problem Definition and Objectives

Problem Definition:-
Despite the advancements in natural language processing, accurately summarizing academic research remains a challenging task due to the technicality and density of content. Most existing summarization models are either extractive in nature or perform poorly when applied to domain-specific texts such as scientific papers. The problem is to develop an abstractive summarization system capable of generating high-quality summaries that are faithful to the original meaning while maintaining linguistic coherence and brevity.

Objectives:-
1. To develop a deep learning-based model for abstractive summarization of research papers using BART and PGN.
2. To pre-process and fine-tune large-scale research paper datasets for training and evaluation.
3.To compare the performance of BART and PGN in terms of summary quality using established metrics like ROUGE and BLEU.
4. To explore the impact of domain-specific language, citation formats, and structure on the summarization quality.
5.To identify the strengths and limitations of each model in the context of summarizing technical, scientific content.

1.4 Project Scope & Limitations

Project Scope:-
This project focuses on summarizing the abstracts and introduction sections of research papers, as they encapsulate key objectives, methods, and conclusions. The system is trained on a curated dataset of scientific texts, primarily from open-access repositories such as arXiv or PubMed, ensuring that the data reflects a wide range of scientific disciplines. The use of both BART and PGN allows us to explore complementary approaches to abstractive summarization: one purely generative and the other hybrid in nature.

Limitations:-
While this project demonstrates the potential of abstractive summarization using BART and Pointer-Generator Networks, it is not without limitations. First, the summarization is limited to the abstracts and introduction sections of research papers, excluding full-document summarization due to computational constraints and model input length restrictions. Second, domain-specific terminology, mathematical expressions, and symbolic notations commonly found in scientific literature can pose challenges for both models, potentially leading to incomplete or inaccurate summaries. Third, the training dataset may have a disciplinary bias—models

trained predominantly on certain fields (e.g., computer science) may not generalize well to others like medicine or physics. Moreover, the quality of generated summaries heavily depends on the quality of training data, and noisy or inconsistently formatted inputs can hinder performance. Finally, standard evaluation metrics like ROUGE and BLEU, while useful, do not always reflect the semantic accuracy or factual consistency of summaries, and the limited scope of human evaluation restricts the depth of qualitative analysis.

1.5 Methodologies of Problem solving

This project follows a comprehensive methodology designed to address the complexity of summarizing academic research papers using deep learning. The process begins with data collection from open-access repositories such as arXiv, PubMed, and S2ORC (Semantic Scholar Open Research Corpus). The focus is on extracting the abstracts and introductions of papers, as these sections typically summarize the core research problem, methodology, and findings. The raw text is then cleaned and preprocessed to remove LaTeX commands, citation markers, reference links, and other non-textual artifacts. Further preprocessing steps include lowercasing, white space normalization, and subword tokenization using Byte-Pair Encoding (BPE) or SentencePiece to efficiently handle vocabulary size and rare words. This processed data is then split into training, validation, and test sets to ensure reliable model evaluation.

For model development, two architectures are employed: BART and the Pointer-Generator Network (PGN). The BART model is fine-tuned using a supervised learning approach, where the model learns to generate summaries from input texts by minimizing the cross-entropy loss between the generated sequence and the ground-truth summary. BART's encoder-decoder transformer architecture allows it to understand the full context of a document and produce fluent, coherent summaries. In contrast, PGN incorporates a hybrid mechanism that combines standard sequence generation with the ability to directly copy words from the input text. This makes PGN particularly useful for handling out-of-vocabulary terms, domain-specific language, and factual references. Both models are trained using GPUs to manage computational demands, and techniques such as gradient clipping, early stopping, and learning rate scheduling are used to maintain stable training and prevent overfitting. After training, the models are evaluated using automatic metrics such as ROUGE-1, ROUGE-2, ROUGE-L, and BLEU, which measure lexical and structural similarity between generated and reference summaries. Additionally, qualitative evaluation is performed on selected outputs to assess coherence, relevance, and readability. A comparative analysis of the results highlights the respective advantages and limitations of BART and PGN, providing insights into their suitability for abstractive summarization in scientific contexts.

CHAPTER 2
LITERATURE SURVEY

In [1], the authors introduce a new neural architecture for abstractive text summarization. Traditional models are plagued by issues such as factual inaccuracies, redundancy, and out-of-vocabulary (OOV) word management. In order to address these issues, the authors introduce a hybrid pointer-generator network that allows the model to create new words as well as copy directly from the source text, which improves accuracy and OOV word management. A coverage component is also implemented to avoid attention history and delete redundancy. Employed on the CNN/Daily Mail dataset, their model achieves more than a 2 ROUGE point gain over previous abstractive models. Despite all its success, the model is nevertheless more extractive than human-produced abstractive summaries and still fails to surpass extractive baselines like the lead-3 method. However, the research is a significant achievement in reliable abstractive summarization and will open the door to future models with additional abstraction levels.

In [2], the authors present an improved Pointer-Generator Network (PGN) for text summarization, made to address long texts more effectively while striking a balance between extractive and abstractive approaches. PGN enables the model to either copy words from the source or create new words, overcoming the drawbacks of basic sequence-to-sequence models, including repetition and factitious errors. To enhance such balance, the authors propose an improved coverage mechanism that better follows attention, diminishes redundancy, and enhances fluency. Additionally, they bring in sophisticated optimization methods, such as reinforcement learning and enhanced loss functions, to enhance coherence as well as factual consistency. Measured on resources such as CNN/DailyMail and Gigaword with respect to ROUGE metrics, the improved model attains superior summary accuracy, readability, and informational retention.

In [3], the authors present an enhanced approach to text and document summarization using neural networks. The proposed model is built on the foundation laid by Pointer-Generator Networks, which is a hybrid model that combines using both abstractive and extractive summarization techniques. The authors use a pointer mechanism to dynamically decide whether to generate a word from its vocabulary or whether to copy a word from the input text. They also explored the use of part of speech features to improve the quality of the generated summaries. The usage of Part of Speech features will also improve word selection, ensuring that key informational elements such as nouns and verbs are properly represented in the summaries. The authors judge the effectiveness of this approach compared to other baseline models using ROUGE score.

In [4], the authors first discuss the effectiveness of multimodal pointer-generator networks in generating summaries that are topically relevant. The authors examine the use of transformer-based architectures that integrate multimodal information such as tabular and visual data to enhance summary generation. The authors then introduce a multimodal pointer-generator transformer model, that not only retains the original PGN model's copy mechanism but also uses pretrained convolutional neural network that integrates the visual representations by extracting the feature embeddings. The authors also employ a multi-head attention mechanism

that can attend to both the textual and the visual features dynamically and assign appropriate weights based on their relevance to the topic mentioned. Evaluation metrics such as ROUGE score and BERTScore are used to judge the summary coherence. Future work involves addressing challenges such as computational overhead, dataset limitations and improving alignment between the modalities.

In [5], the authors propose a method for summarizing spoken conversations by including subject awareness into the Pointer-Generator architecture. The authors mainly aim to address challenges specific to spoken dialogue summarization, such as disfluencies, informal language, and topic shifts that occur due to interruptions and repetitions that make summarization difficult. The authors improve the traditional architecture by integrating topic modeling techniques to guide the generation process. The proposed model contains two main components: a topic aware encoder and a pointer generator decoder, which selectively generates or copies words from the input while ensuring coherence. The results indicate that incorporating topic-awareness improves both lexical diversity and relevance by aligning generated summaries with the core themes of conversations. The authors judge the effectiveness of this approach compared to other baseline models using ROUGE score.

In [6], the authors propose a method for improving sequence-to-sequence (Seq2Seq) pre-training by introducing a Masked Pointer-Generator Network (MAPGN). The proposed model is designed in a way that it addresses the key challenges in sequence to sequence learning to generate better summaries. The authors suggest to integrate a masked sequence pretraining approach with a pointer generation network. This system lets the model use both abstractive and extractive text generating processes. The mechanism works by selecting salient tokens from the input while also generating new tokens where necessary. The training process involves reconstruction of masked portions of input sequences, improving the ability of the model to generate high-quality text. The authors prove that MAPGN significantly improves performance on summarization, translation, and text generation tasks.

In [7], the authors design an approach in which natural descriptions are formed from structured data. The authors introduce Pointer-Generator Network which is an advanced version of the sequence-to-sequence model that includes a traditional generation approach alongside word copying from the input. This machine is very effective for summarization tasks because it aids in the management of out-of-vocabulary terms and improves the factual accuracy of the summary by copying significant phrases from the input. The model has a pointer-based copying mechanism, which, together with a coverage mechanism, allows switching among generation and copying segments of the input text to eliminate repetition in the produced text. The authors perform their evaluation of the model on several benchmark datasets and show its effectiveness in comparison with basic implementations of seq2seq and attention models.

In [8], the authors presented a novel model for text summarization which is an enhancement of the standard Pointer-Generator Network. The authors propose an

architecture for a two-stage encoder combined with pretrained word embeddings to enhance performance in abstractive summarization. The two-stage encoder is designed to first capture important contextual information so that it can be further processed, thus improving feature extraction and summarization. Then, the authors deployed pretrained embeddings like GloVe or Word2Vec for better word representation and to help the summarization model deal with large vocabularies. The two-stage encoder comprises a first encoder which processes and produces the preliminary representations of the input and a second one which improves the coherence and context of the representations. The results the authors provided from the experiments demonstrate that approach surpasses the baseline models in the fluency, fact consistency and the repetition suppression.

In [9], the author focuses on automatically creating research insights from research papers using advanced machine-learning techniques. Research highlights are defined as short, easy-to-understand points summarizing the important findings of a paper. It helps researchers to understand contributions of the paper. The authors used a pointer-generator network combined with SciBERT, a pre-trained language model specifically designed for scientific texts.

In [10], the author introduces a new method to rewrite sentences while keeping their original meaning. This helps to make sentences more diverse, which in turn improves systems like translation tools and question-answering programs. It uses transformer models and word types like verbs, nouns, etc. to understand the sentence structure. The pointer generator Network helps the model to handle rare and unfamiliar words. It is used to create sentences that appear like the original, but they are different from the original text. they both have similar meaning.

In [11], the author introduces a new model for automatic text summarization using a Double Attention Pointer Network. This model improves the quality of generated summaries by combining self-attention and soft attention with a pointer network which in turn helps in reducing repetition. It reduces errors during training using scheduled sampling and reinforcement learning. It is better as compared to state-of-the-art models.

In [12], the authors explore the effectiveness of Google's Flan-T5 model for abstractive text summarization. They evaluate the model's performance across various newsfocused datasets and illustrate the challenges in accurately assessing AI-generated summaries. The study also involves fine-tuning Flan-T5's configuration—such as sequence length and beam search settings—to balance summary quality with computational efficiency.

In [13], the author categorizes summarization techniques into two main types: single-document and multi-document summarization. The paper discusses three methods—Word Graph-based Method, which produces syntactically valid sentences without semantic understanding; Sentiment Infusion Method; and the Semantic Graph Reduction Approach for single-document summarization. To address overlapping and redundant content, the study introduces techniques such as

the Genetic Semantic Graph and the Clustered Genetic Semantic Graph, which aim to reduce redundancy while preserving unique and relevant information.

In [14], the authors explore the use of multimodal techniques to enhance abstractive text summarization. Traditional text-only methods often struggle to fully capture semantic context and sentence-level relationships. To overcome these challenges, the study introduces a Multimodality Image Text (MIT) layer that combines text embeddings with image features, integrated into a sequenceto-sequence (seq2seq) architecture using attention mechanisms and LSTM. Experiments conducted on the MSMO dataset demonstrated superior performance compared to existing approaches, with the model achieving high ROUGE scores (ROUGE-1 = 52.33, ROUGE-2 = 34.18), indicating improved summary quality and relevance.

In [15], the author provides a comprehensive survey of Automatic Text Summarization (ATS), highlighting key challenges such as content selection, coherence, and redundancy. Although significant advancements have been made, generating human-like summaries remains a complex task. ATS is particularly vital in areas like search engines, news aggregation, and big data analytics, especially in realtime, domain-specific applications such as legal and medical fields. The paper reviews major datasets and evaluation metrics, including ROUGE and BLEU, discusses ethical concerns like bias, and emphasizes the need for further research to improve evaluation methods and abstractive techniques, encouraging interdisciplinary collaboration.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

Assumptions:-
1. Input Papers Are in English:The summarization system assumes that all research papers are written in English, as the models are trained primarily on English-language data and may not perform well on multilingual inputs.
2. Clean and Preprocessed Input Text: It is assumed that the input text (abstracts or introductions) is free of major formatting errors, LaTeX artifacts, or incomplete sentences, which could otherwise affect the summarization quality.
3. Availability of Annotated Summaries: For supervised training, it is assumed that each input text comes with a corresponding human-written summary or abstract that serves as ground truth during model training and evaluation.

Dependencies:-
1. Pre-trained NLP Models and Libraries: The project depends on Hugging Face's Transformers library for access to pre-trained models like BART, and on custom implementations or extensions of the Pointer-Generator Network. Proper functioning requires compatible versions of these libraries.
2. Hardware Acceleration (GPU/TPU Support): Training and fine-tuning deep learning models require high-performance hardware, such as NVIDIA GPUs or Google TPUs.
3. Datasets for Training and Evaluation: The system relies on publicly available summarization datasets such as CNN/DailyMail, Gigaword, and possibly domain-specific corpora like arXiv or S2ORC for training and evaluation.

3.2 Functional Requirements

3.2.1 System Feature 1: Text Preprocessing

Description:-
The system must be able to process raw research papers by extracting the relevant text from documents. It should clean and prepare the text for further processing, removing any irrelevant information.

Requirements:-
1. The system should extract abstract and introduction sections from research papers in formats like PDF, Word, or LaTeX.
2. The input text should be normalized (e.g., converting to lowercase, removing extra spaces).
3. Tokenization and subword encoding (e.g., Byte-Pair Encoding or SentencePiece) should be applied to handle complex vocabulary.

3.2.2 System Feature 2: Abstractive Summarization Model

Description:-
The core functionality of the system is to generate abstractive summaries from the

preprocessed input text using deep learning models such as BART and Pointer-Generator Networks (PGN). These models must generate fluent, concise, and informative summaries of research papers' abstracts and introductions.

Requirements:-
1. The model should generate summaries that preserve key information while being concise and fluent.
2. The system must allow for fine-tuning on domain-specific datasets for improved summarization of technical or scientific content.
3. The summarization models should incorporate a balance between copying and generating text to handle domain-specific terminology.

3.2.3 System Feature 3: Data Storage and Management

Description:-
The system must store input research papers, their corresponding summaries, and evaluation results for future reference and analysis.

Requirements:-
1. The system must store input papers and their corresponding summaries securely in a database.
2. The system must log evaluation scores and feedback for each summary generated.
3.The system should allow for easy retrieval of stored summaries for comparison and further analysis.

3.3 External Interface Requirements

3.3.1 User Interfaces

Feature	Description
Application Interface	A user-friendly interface designed for easy navigation of the website.

Table 3.1: User Interfaces

3.3.2 Software Interfaces

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. Response Time: The system should generate summaries within a reasonable time frame—ideally under 10 seconds for short to medium-length inputs on a

Software Interface	Description
Database Management System	SQLite for storing user data, uploaded papers, and generated summaries.

Table 3.2: Software Interfaces

GPU-enabled setup.
2. Scalability: The system should be capable of scaling to handle multiple summarization requests concurrently, especially when deployed on cloud infrastructure or used in batch processing scenarios.
3. Model Efficiency: The summarization models should be optimized to reduce memory usage and computational load without compromising summary quality.
4. Throughput: The system should be able to process and summarize multiple research papers in a queue-based manner, enabling batch-mode summarization for larger datasets.

3.4.2 Usability and Maintainability Requirements

1. User-Friendly Interface: The interface should be intuitive, allowing users to easily upload papers, view summaries, and understand evaluation scores without requiring technical knowledge.
2. Modular Design: The system should follow a modular architecture, allowing individual components to be updated or replaced independently.
3. Error Handling and Feedback:The system should provide meaningful error messages and guidance when an input cannot be processed.
4. Documentation and Maintainability: The system should be well-documented with clear instructions for setup, usage, and model retraining, making it easier for future developers or researchers to maintain and extend the system.

3.5 System Requirements

3.5.1 Hardware Requirements

1. Processor: A multi-core CPU (Intel i7/Ryzen 7 or higher) is recommended for efficient data preprocessing and basic model operations.
2. Memory (RAM): Minimum 16 GB RAM is required for smooth model training and handling large datasets during preprocessing.
3. Graphics Processing Unit (GPU): At least one NVIDIA GPU (e.g., RTX 3060 or better, with 8GB+ VRAM) is recommended for training and inference using BART and PGN models.

3.5.2 Software Requirements

1. Operating System: Ubuntu 20.04 LTS or Windows 10/11 (64-bit) with support for Python development and CUDA (for GPU use).
2. Programming Language: Python 3.8+ with relevant packages for NLP and deep learning (e.g., transformers, torch, tensorflow, nltk, spacy).
3. Frameworks & Libraries: Hugging Face Transformers, PyTorch or TensorFlow, and additional tools like scikit-learn, matplotlib, and ROUGE or NLTK for evaluation.

3.5.3 Network Requirements

1. Internet Connection: A stable internet connection is required to download pretrained models, datasets, and software packages during setup.
2. Cloud Services (Optional): If running in a cloud environment (e.g., Google Colab, AWS, or GCP), access to cloud-based GPUs or TPUs may be required.

3.6 Analysis Models: Agile Methodology to be applied

For the successful execution of this project, the Agile Software Development Methodology has been adopted. Agile is an iterative and incremental approach that emphasizes flexibility, customer collaboration, and rapid delivery of functional software. This methodology is particularly well-suited to machine learning and NLP projects, where frequent experimentation and feedback-driven development are essential. The Principles from the Agile Development Model are:

1. Incremental Development: The project was divided into multiple sprints, each focusing on a specific functional block—such as data preprocessing, model integration (BART and PGN), evaluation, and UI development.
2. Continuous Feedback and Iteration: Agile encourages short feedback loops. After each sprint, the output was evaluated using test cases and validation metrics.
3. Collaboration and Flexibility: Agile emphasizes adaptability. Requirements were refined continuously based on new insights (e.g., difficulty in preprocessing certain documents or the need to adjust training parameters). This flexibility was crucial in model selection and tuning.
4. Prioritized Backlog: A product backlog was maintained to track pending tasks, such as integrating evaluation metrics, refining the UI, or enabling document upload.

This adaptable approach will facilitate rapid changes and continuous improvement throughout the project lifecycle.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

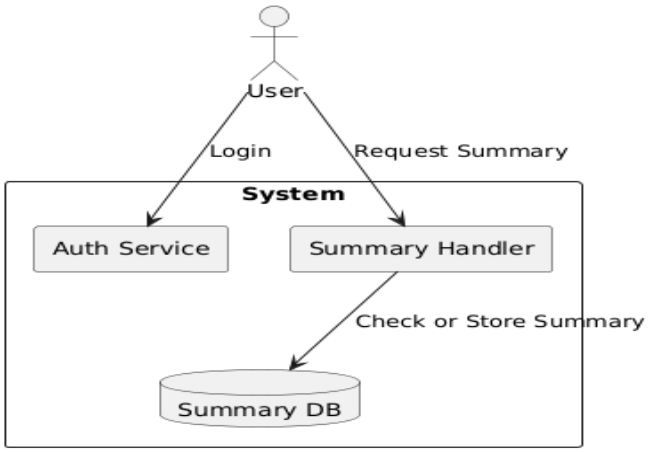


Figure 4.1: System Architecture

The System Architecture Diagram involves various components:-
1. Users:
The user interacts with the system through two primary actions:
a) Login: To authenticate themselves before accessing the system.
b) Request Summary: To request an abstractive summary for a research paper.

2. Core System Components:
a) Auth Service:
This handles user authentication. When a user logs in, their credentials are verified through this service. Ensures secure access to the system.
b) Summary Handler:
This is the core logic component responsible for handling summary requests. Once the user is authenticated, summary requests are sent to this handler. It coordinates with the database to check if a summary already exists or if a new one needs to be generated.

3. Summary DB:
The Summary Database stores previously generated summaries and possibly user-specific data. The Summary Handler checks this database:
a) If the summary exists, it retrieves it.
b) If not, it may initiate the summarization process and then store the result.

4. Flow of Operations:
a) User logs in → Auth Service authenticates.
b) User requests summary → Summary Handler receives the request.
c) Summary Handler queries Summary DB:
i) If found: return the existing summary.
ii) If not found: trigger summarization logic (e.g., BART + PGN), then store the result.

4.2 Mathematical Model

4.2.1 Mathematical Model

Let the system be represented as a tuple:

$$S = \{I, P, O, F, D, A, R\}$$

Where:

- **I = Input Set**
  I = {Raw_PDF, User_Query, MetaData, Access_Credentials}
  Inputs include raw research paper PDFs, user queries for summarization, metadata (e.g., title, author), and access credentials for secure access.

- **P = Set of Processes**

  P = {Text_Extraction, Preprocessing, BART_Inference, PGN_Reranking, Storage, Visualization}

  Processes involve extracting text from PDFs, preprocessing for model input, summarization using BART, refinement with PGN, storing summaries, and rendering results.

- **O = Output Set**
  O = {Abstractive_Summary, Visualization, Alerts, UserLogs}
  Outputs include the generated summary, visual analytics, alerts for summarization failures, and logs for auditing.

- **F = Set of Functions**
  $F = \{f_1, f_2, f_3, f_4, f_5\}$ where:
  f_1: Extract text and metadata from PDFs
  f_2: Preprocess content for model readiness
  f_3: Generate abstractive summary using BART
  f_4: Apply PGN to enhance or refine output
  f_5: Display results and update logs

- **D = Database**
  D = {MongoDB}
  Stores extracted text, user queries, generated summaries, and feedback logs.

- **A = Actors**
  A = {User, Admin, SummarizationEngine}
  Actors include the end-user requesting summaries, system administrators, and the backend engine running the models.

- **R = Set of Rules/Conditions**

  $$R = \{ \begin{array}{l} \text{if PDF format unsupported} \rightarrow \text{throw format error,} \\ \text{if summary length} > \text{threshold} \rightarrow \text{rerun PGN,} \\ \text{if user request fails} \rightarrow \text{notify admin via alert,} \\ \text{if model confidence low} \rightarrow \text{log for review} \end{array} \}$$

4.3 Data Flow Diagrams

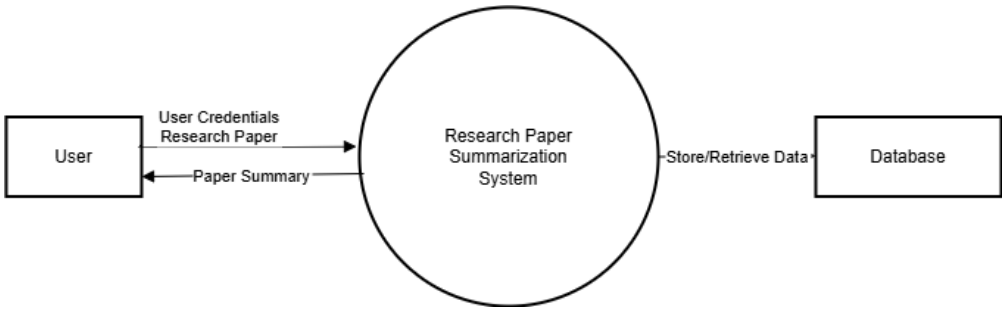


Figure 4.2: DFD Level 0 – Context Level

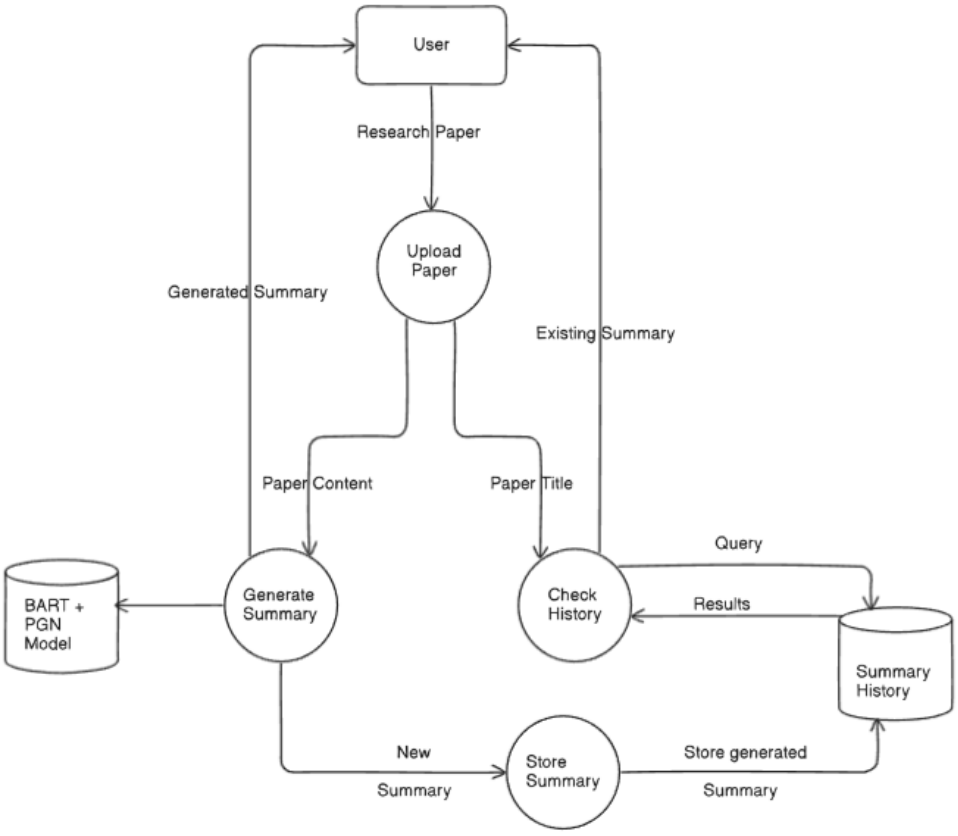


Figure 4.3: DFD Level 1 – Detailed View

4.4 Use Case Diagram

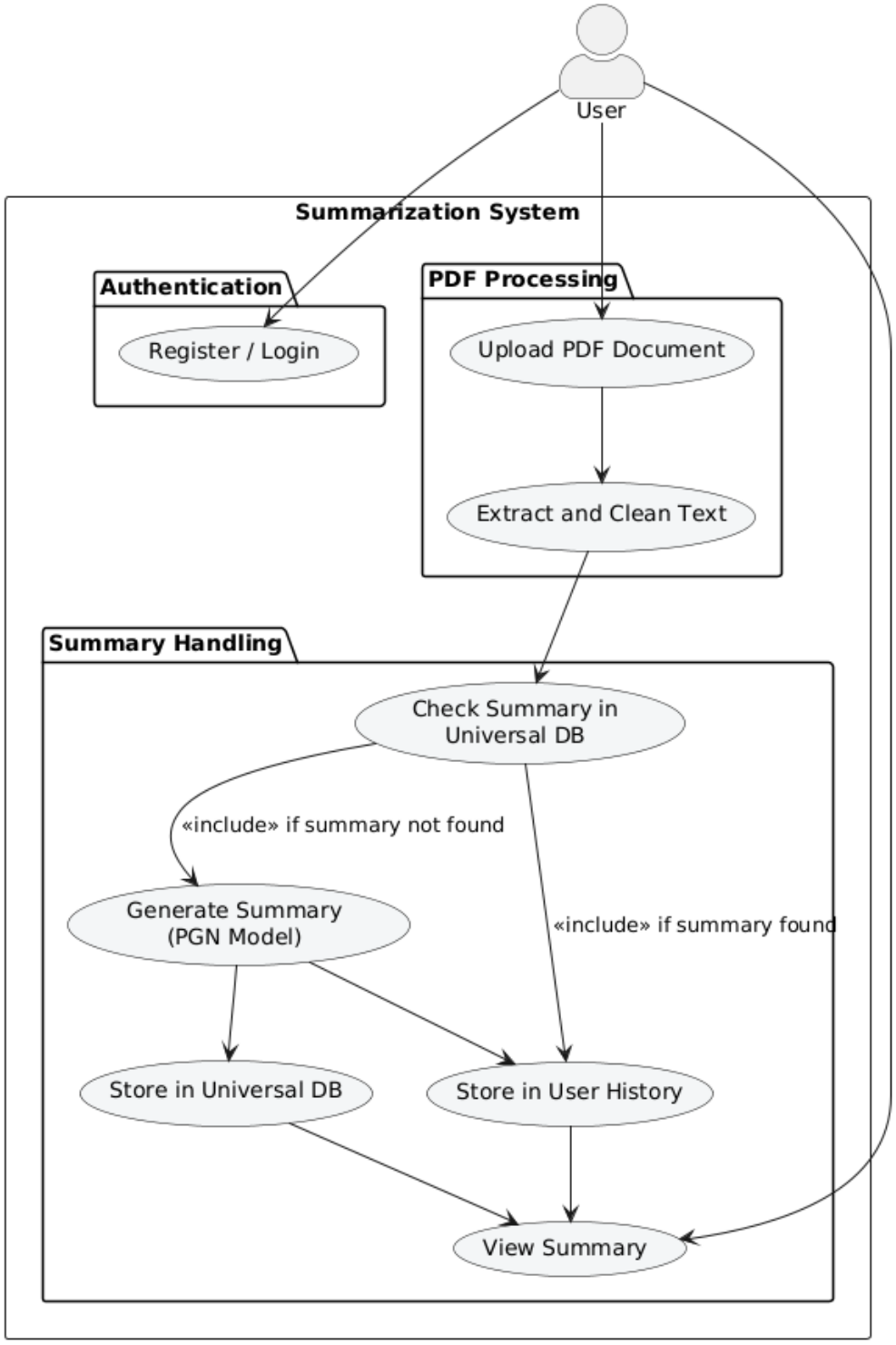


Figure 4.4: Use Case Diagram

4.5 Entity Relationship Diagram

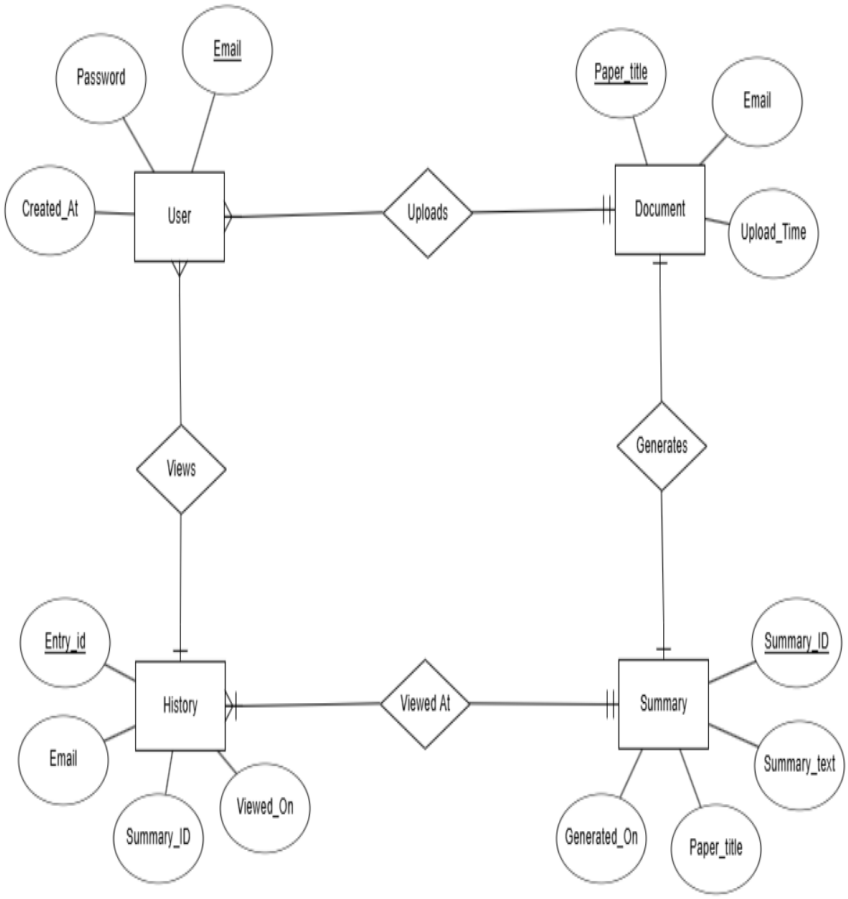


Figure 4.5: Entity Relationship Diagram

4.6 Class Diagram

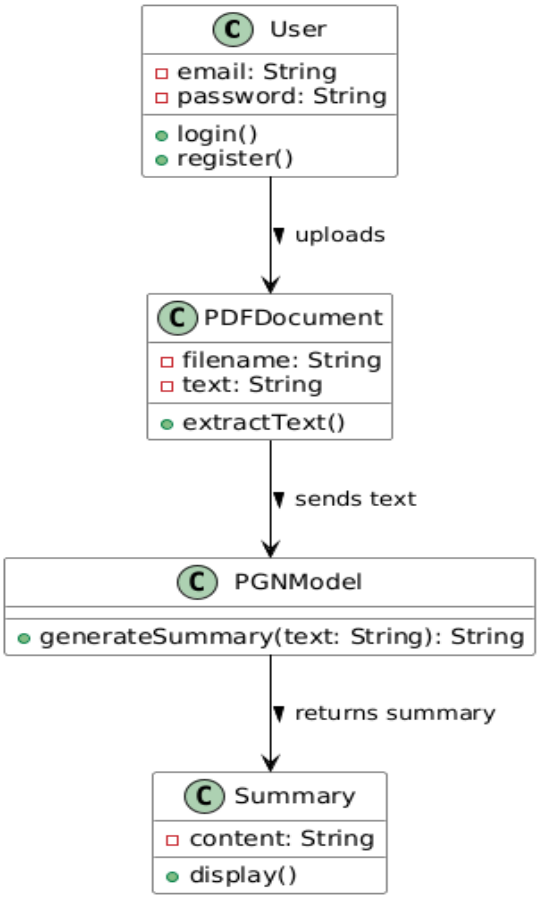


Figure 4.6: Class Diagram

4.7 Sequence Diagram

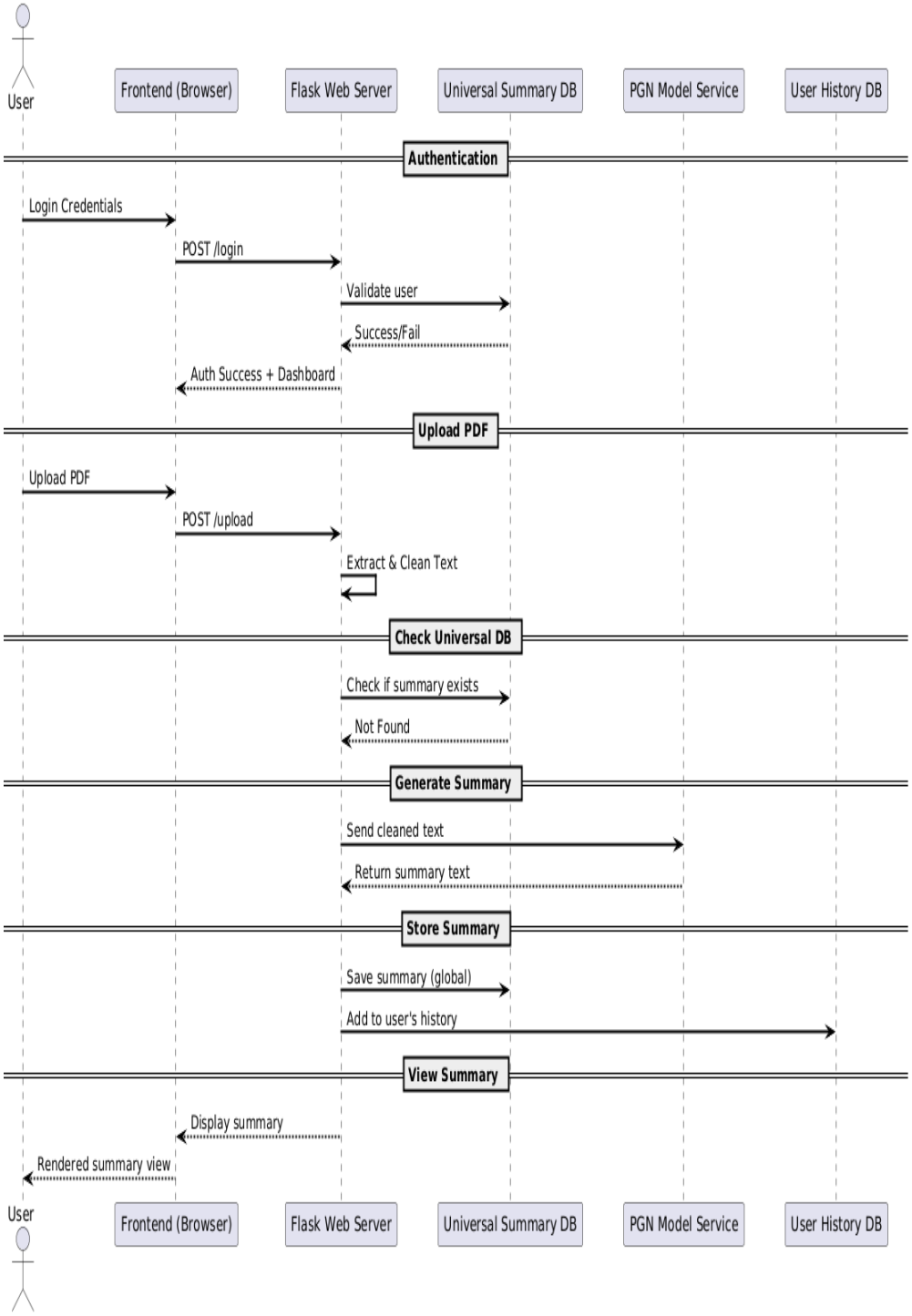


Figure 4.7: Sequence Diagram

4.8 Deployment Diagram

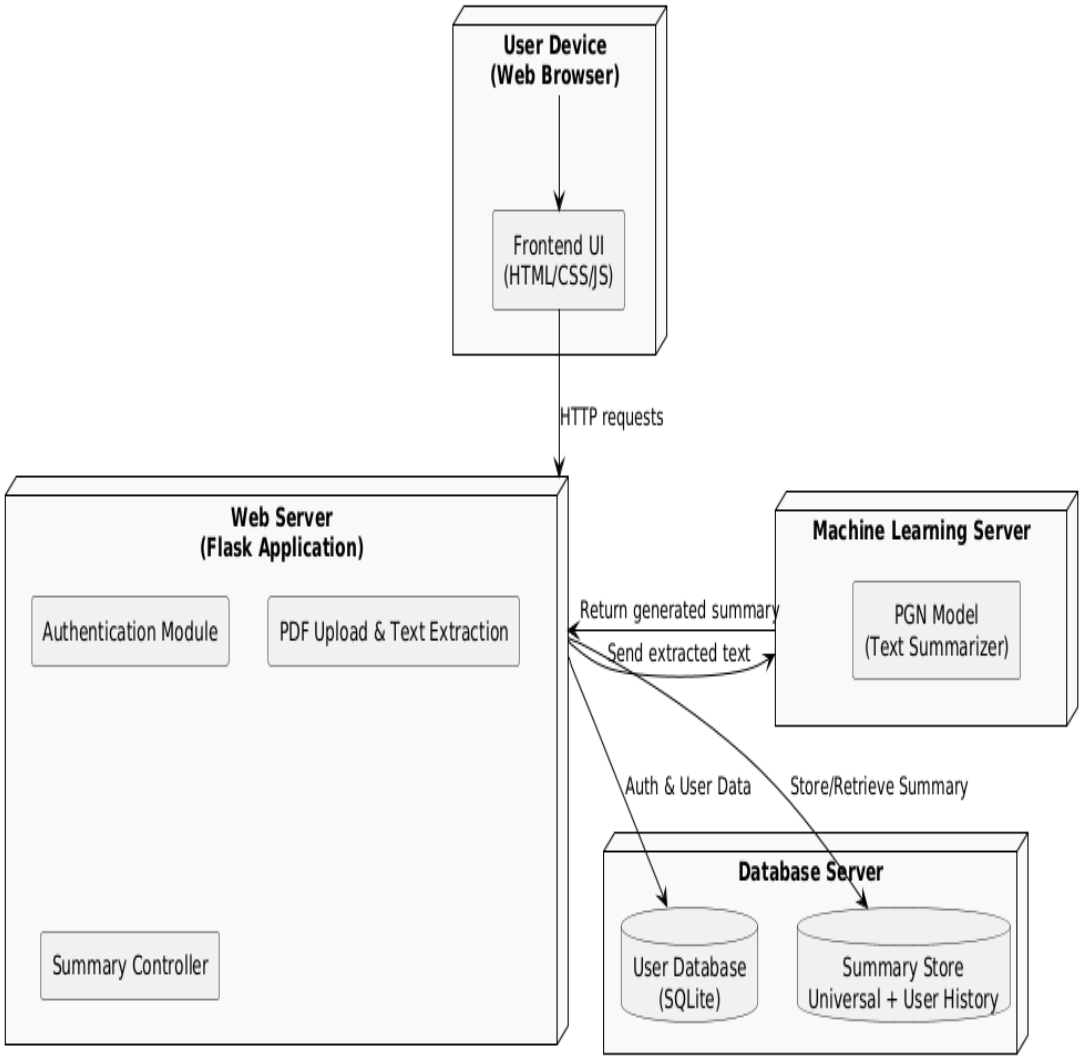


Figure 4.8: Deployment Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

1. Research and Development phase: Estimated at 6 months, focuses on building core features like This includes phases such as requirement gathering, model development, evaluation, UI creation, and documentation. In this phase, we get insights on efficient resource allocation while ensuring a user-friendly interface to streamline operations and improve system performance.
2. Testing and Evaluation: Estimated at 2 months, involves performance, security, and user acceptance testing. These tests will ensure efficient summary evaluation, fine-tuning models, and user feedback analysis validating the system's core functionalities.
3. Documentation and Review: Estimated at 1.5 months, will include preparing project reports, user manuals, and presentation materials. This ensures the system's usability, accuracy, and alignment with normal user needs.

5.1.2 Project Resources

1. Frontend Developer

- Responsible for designing and developing the user interface using HTML, CSS, JavaScript, and React.js.
- Uses technologies like HTML, CSS, JavaScript, and frameworks like React or Streamlit to create pages such as Home, Login, Register, About Us, and History.
- Will ensure proper interaction with backend APIs and renders summary results clearly for users.

2. Backend Developer

- Handles server-side logic and API creation using Flask or FastAPI.
- Handle data processing, server-side logic, and integration with the frontend.
- Responsible for integrating the summarization models (BART, PGN), managing user authentication, routing, and providing endpoints for uploading papers, processing text, and storing/retrieving summaries.

3. Database Technology

- Utilize SQLite for efficient data storage and retrieval, enabling flexible schema design.
- Ensure proper data modeling to support storing and managing user data (login credentials, history of summarized papers, and performance metrics).
- Implement aggregation queries to analyze data and generate insights for users.

4. Collaboration Tools

- Use GitHub for version control, enabling collaboration and tracking code changes.
- Implement Google Meet for team communication and real-time collaboration during development.
- Utilize project management tools (e.g., Trello or Jira) for task assignment and progress tracking.

5.2 Risk Management

Risk management is essential for the successful implementation of the abstractive summarization application. By identifying potential risks early, we can mitigate their impact on development and deployment.

1. **Technical Risks:** Integration challenges between technologies like Flask, HTML and SQLite may arise. To mitigate this, thorough testing will be conducted at each development phase.
2. **Data Security Risks:** Improper handling of user-uploaded documents can lead to data leaks or unauthorized access if authentication is weak.
3. **User Acceptance Risks:** Users may not find the summaries sufficiently readable or coherent if the model's performance is not properly tuned. User feedback will be gathered during testing to refine the interface and features.
4. **Project Timeline Risks:** Delays due to unforeseen challenges could impact the timeline. We will adopt an agile approach with regular progress reviews to address issues promptly.

By proactively managing these risks, we aim to enhance the project's reliability and ensure a smooth deployment of the application.

5.2.1 Risk Identification

Risk identification is crucial for anticipating potential issues that could impact the abstractive summarization application's development and deployment. The following risks have been identified:

- **Technical Integration Risks:** Challenges in connecting frontend, backend, and model inference pipeline could lead to functionality issues.
- **Data Security Risks:** These risks refer to the exposure of sensitive documents or summaries due to poor encryption/authentication.
- **User Experience Risks:** The application may not meet the usability needs of the users due to ineffective summarization leading to dissatisfaction or confusion.

- **Performance Risks:** Slow response time due to heavy model computation on limited hardware, affecting user experience and system responsiveness.
- **Resource Risks:** Limited availability of skilled developers or unforeseen team changes may delay project timelines and impact quality.
- **Compliance Risks:** Failure to comply with data privacy norms for storing user-uploaded content could lead to legal issues and fines.
- **Change Management Risks:** Difficulty adapting to scope changes or incorporating new features late in the cycle could hinder successful implementation and usage.

5.2.2 Risk Analysis

Risk analysis involves evaluating the identified risks to determine their potential impact on the lab management desktop application project and the likelihood of their occurrence. This process helps prioritize risks and inform mitigation strategies.

- **Technical Integration Risks:**
  - **Impact:** High – If integration issues occur, it can halt development and delay the project.
  - **Likelihood:** Moderate – While integration challenges are common, careful planning and testing can reduce the risk.
- **Data Security Risks:**
  - **Impact:** Critical – A data breach may lead to user distrust, data loss, or privacy breaches.
  - **Likelihood:** Moderate – With proper security measures, the risk can be minimized, but vulnerabilities may still exist.
- **User Experience Risks:**
  - **Impact:** Moderate – Poor summaries can result in user churn and negative feedback, which will in turn lead to low adoption and decreased productivity.
  - **Likelihood:** High – User feedback during development can mitigate this risk, but misalignment with user needs may still occur.
- **Performance Risks:**
  - **Impact:** High – May hinder adoption due to high latency or model crashes under load.
  - **Likelihood:** Moderate – The risk can be managed through optimization and performance testing.
- **Resource Risks:**

  - **Impact:** Moderate – Lack of computing power or skilled personnel may limit training and performance.
  - **Likelihood:** High – Staffing changes or resource limitations are common in projects, making this a significant risk.

- **Compliance Risks:**
  - **Impact:** Critical – Can lead to project rejection in academic or enterprise settings due to non-compliance.
  - **Likelihood:** Low – With adequate legal oversight, compliance issues can be minimized.

- **Change Management Risks:**
  - **Impact:** Moderate – Scope changes without proper versioning may introduce bugs and inconsistencies.
  - **Likelihood:** Medium – Proactive training and communication can alleviate resistance, but some pushback is likely.

5.3 Project Schedule

5.3.1 Project Task Set

1. **Conduct a Needs Assessment:** Identify the core challenges faced by students and researchers in summarizing long research papers through surveys and informal feedback.
2. **Meet with Stakeholders:** Interact with faculty members, research scholars, and end users to gather expectations, use cases, and desired functionalities of the summarization system.
3. **Formal Discussions:** Discuss with academic supervisors or mentors to align project goals with educational and research standards.
4. **Brainstorming Session:** Collaborate with the development team to brainstorm solutions based on the gathered requirements.
5. **Documentation:** Prepare structured documentation detailing the problem statement, scope, objectives, and collected user requirements.
6. **Technology Stack Identification:** Evaluate and select the technology stack, including frontend and backend technologies, based on feasibility and project requirements.
7. **Research Innovative Solutions:** Explore advanced summarization techniques, including hybrid models, attention mechanisms, and decoding strategies.

8. **Team Assignments:** Assign specific modules and tasks to team members based on their expertise and experience.
9. **Learning Phase:** Get hands-on with relevant technologies and libraries, such as BART, the Pointer-Generator Network, ROUGE metrics, and model deployment practices.
10. **Design User Interfaces:** Create interactive and user-friendly interface designs for the application.
11. **Design Approval:** Review and finalize the UI/UX design with feedback from users and mentors before moving to implementation.
12. **Development Phase:** Develop user interfaces and functional modules concurrently to streamline the development process.
13. **Model Identification:** Choose pre-trained models (e.g., BART) and implement the improved PGN with enhancements such as coverage mechanism or reinforcement learning.
14. **Model Development and Testing:** Fine-tune the models on datasets like CNN/DailyMail or scientific corpora, then evaluate using ROUGE scores and qualitative review.
15. **Integration Phase:** Seamlessly integrate frontend, backend, and the summarization models into a unified application.
16. **Deployment:** Deploy the application on a cloud service or local server, ensuring performance, reliability, and ease of access.
17. **Iterative Improvement:** Refine model outputs, interface usability, and backend performance based on testing and real user feedback.
18. **User Testing and Feedback:** Conduct evaluation sessions with actual users (students, researchers) to assess output quality and usability.
19. **Documentation:** Compile the complete documentation including architecture, codebase, user manual, model details, and evaluation results.
20. **Project Submission:** Finalize and submit the entire project with working demo, documentation, and reports for academic and future use.

5.3.2 Timeline Chart

This section represents a summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

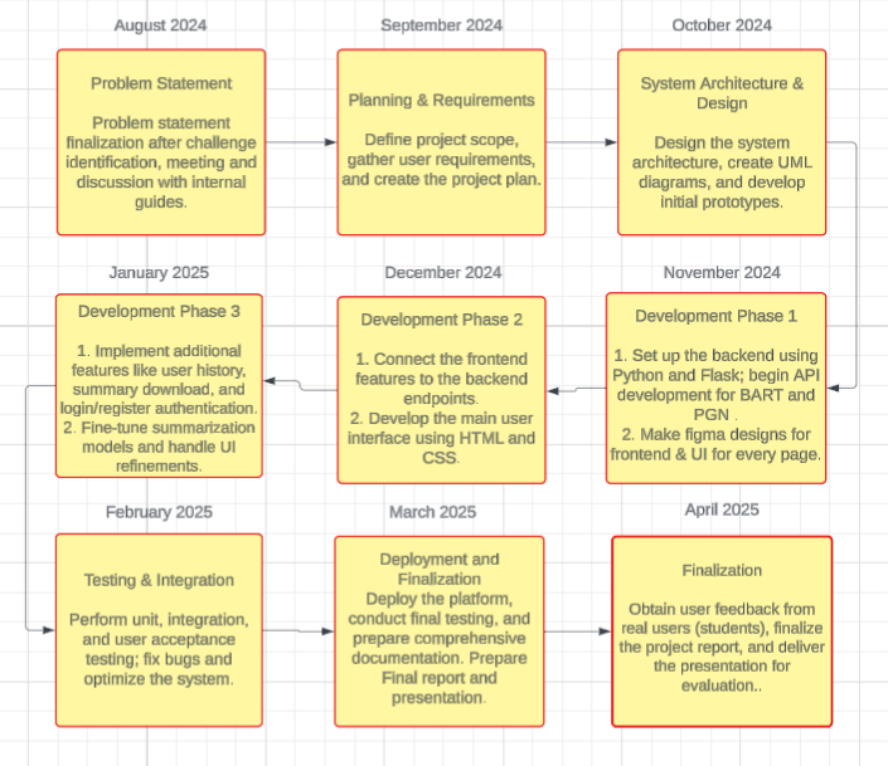


Figure 5.1: Summarized TimeLine Chart

5.4 Team Organization

5.4.1 Team structure

- **Ajit Abhyankar:** Backend Developer, UI/UX Designer.
- **Aniket Rajesh:** Frontend Developer, System Designer.
- **Vedant Kokane:** Cloud Engineer, Backend Developer.
- **Vivek Gotecha:** Frontend Developer, Documentarian.
- **Dr. B. A. Sonkamble:** Internal Mentor and Guide.

5.4.2 Management reporting and communication

Management

For management during our project, we used several effective tools and practices to ensure regular and timely updates. We used **Google Groups** for team communication, where we regularly shared updates, research papers, and important documents. Additionally, we collected various research papers related to our project and organized as well as structured them properly within **Google Drive**. This made it easy for the entire team to access and refer to the collected material. For code management and version control, we used **Git** and **GitHub**, ensuring that all team members could collaborate, track changes, and contribute to the project.

Reporting
Bi-weekly, we reported our progress to the internal mentor, and provided updates on work completed, challenges faced, and ideas we came up with. The regular communication allowed us to stay aligned with project goals and deadlines of submissions. We consulted our advisor **Dr. P. T. Kulkarni** for new ideas and strategic guidance, which we will be implementing in our project, ensuring improvements and innovation.

Communication
For regular communication with team members, we used **Google Groups** and **Whatsapp** for quick updates, making it easy to share new information and get immediate feedback. Our communication with the mentor was structured around bi-weekly sessions and regular updates, with tasks given accordingly.

Thus, this structured approach to management, reporting, and communication helped us to maintain clarity, accountability, and efficiency throughout the project.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The Summary Generator system comprises the following core modules:

- **Data Preprocessing Module:** This module is responsible for preparing the textual data before it is input into the model. The CNN/DailyMail dataset, a benchmark dataset for summarization tasks, is used.This preprocessing ensures that the data is well-structured and model-ready, reducing errors during training and inference.
- **Summarization Engine:** At the heart of the system lies the BART-large model, a pretrained encoder-decoder transformer from Hugging Face's Transformers library. It is especially well-suited for sequence-to-sequence tasks such as text summarization.
- **Training Module:** This module defines the training strategy and manages the learning process. This also includes Loss functions while ignoring padding tokens to compute the difference between the predicted and actual summaries.
- **Evaluation Module:** To evaluate how well the generated summaries match human-written references, this module employs ROUGE metrics. These metrics provide an objective, standardized way to evaluate the model's performance and help fine-tune its architecture or training parameters.
- **Inference and Beam Search Decoding Module:** This module allows the model to generate summaries from new, unseen text inputs and ensures the outputs are of high quality.
- **UI Module:** A User Interface that allows users to upload their research papers and view their histories which includes previously uploaded papers and their generated summaries.

6.2 Tools and Technologies Used

The following tools and technologies were employed to develop the Electron Eye system:

- **Frontend:** HTML – For building an easily navigable and user-friendly interface.
- **Backend:** Flask – For loading models, processing data, and communicating with the database.
- **Database:** SQLite – Used for efficient and schema-flexible data storage.
- **Libraries and Frameworks:** HuggingFace for BART and tokenization, PyTorch for model architecture,training and inference, ROUGE for summary evaluation.

- **Datasets:** CNN/DailyMail – For initial training of the model followed by finetuning on the SciSummNet dataset.
- **Version Control:** GitHub – For source code management and collaboration.
- **Communication and Coordination:** Google Meet, Trello – For team meetings, task tracking, and agile project management.

6.3 Algorithm Details

6.3.1 Algorithm 1: Data Collection Algorithm

- **Dataset Loading:** The CNN/DailyMail dataset is loaded using Hugging Face's datasets library. Each sample consists of an article (input) and highlights (target summary).
- **Cleaning:** Whitespace normalization and removal of unwanted symbols or escape characters.
- **Tokenization:** The BART tokenizer splits the input and summary into sub-word tokens and converts them into corresponding token IDs. It also generates attention masks which help the model focus on meaningful tokens.
- **Truncation and Padding:** Truncation ensures that input text longer than a fixed length is clipped. Padding adds special tokens (¡pad¿) to shorter sequences so all inputs in a batch are the same length.
- **Preparation for Batching:** A custom collate function is defined to shift target summaries for teacher forcing, ensuring the decoder gets previous tokens as inputs during training.

6.3.2 Algorithm 2: BART with Pointer Generator Integration

- **Base Model:** The BART-large model is used as the base. It provides strong performance in sequence-to-sequence tasks.
- **Pointer Generator Mechanism:** Introduce a feedforward layer that takes the decoder hidden state and attention context vector as input.
- **Final Output Distribution:** This allows the model to either generate a token or directly copy it from the input based on context.

6.3.3 Algorithm 3: Model Training and Loss Computation

- **DataLoader:** Batched input is prepared using PyTorch's DataLoader, which loads and pads inputs efficiently.

- **Teacher Forcing:** During training, the decoder is fed the actual ground-truth summary tokens (shifted by one timestep).
- **Loss Function:** CrossEntropyLoss is used to calculate the difference between predicted tokens and actual target tokens. Padding tokens are masked to avoid contributing to the loss.
- **Optimizer:** AdamW is used for optimization with weight decay to prevent overfitting.
- **Learning Rate Scheduler:** A linear scheduler gradually increases and then decreases the learning rate, helping the model converge more smoothly.

6.3.4 Algorithm 4: Evaluation using ROUGE Metrics

- **Model Evaluation:** The trained model generates summaries on a held-out validation set.
- **Metric Calculation:** The model evaluation is carried out on the basis of how well the generated summaries match the human-written references.
- **Result Interpretation:** Higher ROUGE scores indicate better preservation of meaning and structure from the original text.

6.3.5 Algorithm 5: Inference and Beam Search Decoding

- **Input Handling:** Accept user-provided text (e.g., abstract or section from a research paper).
- **Tokenization:** The text is tokenized using the BART tokenizer to produce input IDs and attention masks.
- **Beam Search Decoding:** Instead of selecting the most probable word at each step (greedy decoding), beam search considers multiple paths.A beam width (e.g., 4) is used to maintain the top N most likely sequences. This reduces the risk of generic or repetitive summaries.
- **Summary Output:** The best scoring sequence is decoded back into human-readable text using the tokenizer's decode function and returned as the final summary.

CHAPTER 7
SOFTWARE TESTING

7.1 Types of Testing

The following types of testing were conducted to ensure the robustness and reliability of the abstractive summarization application:

- **Unit Testing:** Unit testing was performed on individual components such as the text preprocessing module, tokenizer functions, file upload handlers, and backend API endpoints. This ensured that each function worked as intended in isolation.
- **Integration Testing:** Integration testing focused on verifying that different modules of the system—such as the frontend, backend API, and summarization models—worked together seamlessly.
- **System Testing:** In system testing, the complete application was tested end-to-end to validate its compliance with the original requirements. This included logging in, uploading a paper, generating a summary, and retrieving history.
- **Functional Testing:** Functional testing focused on verifying that the core features of the application—such as file upload, summary generation, login/register, history display, and evaluation feedback—worked correctly. It ensured that the output of each function matched the requirements and user expectations.
- **Security Testing:** Security testing was conducted to validate user authentication and data protection. Tests included checking for secure password storage, prevention of unauthorized access, validation of user sessions, and protection against injection attacks in form fields and API endpoints.
- **Performance Testing:** Performance testing evaluated the response time and efficiency of the summarization system, especially under load. Metrics such as summary generation time, database access time, and UI rendering were measured.
- **User Acceptance Testing (UAT):** UAT was performed with real users—students and researchers—to assess usability, accuracy, and satisfaction. Feedback was collected on the quality of summaries, ease of navigation, and overall experience.

7.2 Test cases & Test Results

Below are representative test cases along with their expected and actual outcomes during the testing phase:

- **Test Case 1: Input Text Preprocessing**
  - **Input:** A raw research paper in PDF format.

  - **Expected Output:** The text is extracted correctly and preprocessed (removal of stop words, punctuation, etc.).
  - **Actual Result:**Text extracted and preprocessed successfully, ready for summarization. **(Pass)**

- **Test Case 2: Summarization Output Length**
  - **Input:** Research paper with 1500 words.
  - **Expected Output:** The generated summary is approximately 200-300 words, maintaining the key points.
  - **Actual Result:** Summary length within the expected range, with key points preserved. **(Pass)**

- **Test Case 3: Coherence and Readability of Summary**
  - **Input:** A research paper on Natural Language Processing.
  - **Expected Output:** The generated summary should maintain logical coherence, readability, and should not have any grammatical errors.
  - **Actual Result:** Summary coherent, with minor improvements needed in sentence structure. **(Pass)**

- **Test Case 4: Handling Complex Sentences**
  - **Input:** Research paper with long, complex sentences (e.g., multiple clauses).
  - **Expected Output:** The model should correctly identify the core meaning and simplify the sentence structure while maintaining key information.
  - **Actual Result:** Complex sentences are simplified and core meaning retained. **(Pass)**

- **Test Case 5: Summarization with BART + PGN Integration**
  - **Input:** Research paper with a highly technical focus.
  - **Expected Output:** The summary should capture technical jargon accurately and retain essential concepts.
  - **Actual Result:** Key technical concepts captured and summarized correctly. **(Pass)**

- **Test Case 6: Handling Unsupported File Formats**
  - **Input:** Non-PDF file.
  - **Expected Output:** System should detect the unsupported format and prompt the user to upload a PDF.
  - **Actual Result:** Unsupported file format handled correctly with error message. **(Pass)**

CHAPTER 8

RESULTS

8.1 Outcomes

- We successfully developed a system that accurately generates concise and coherent summaries of research papers using BART and PGN, helping users quickly grasp complex content.
- We successfully integrated the advanced summarization model into a web-based interface, making it accessible and easy to use for both technical and non-technical users.This outcome demonstrates that cutting-edge machine learning solutions can be made practical and accessible when paired with intuitive frontend design and efficient backend processing.
- The project has resulted in a modular architecture that can be extended or scaled for future needs. The framework is designed to support the integration of additional models, language support, and data sources.

8.2 Screen Shots

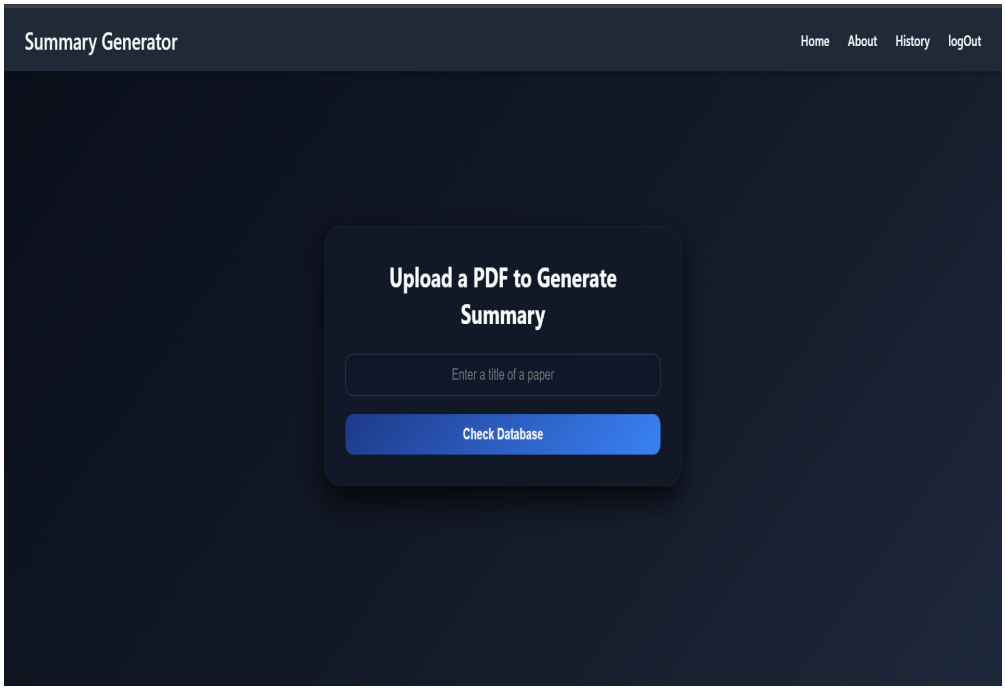


Figure 8.1: Home Page

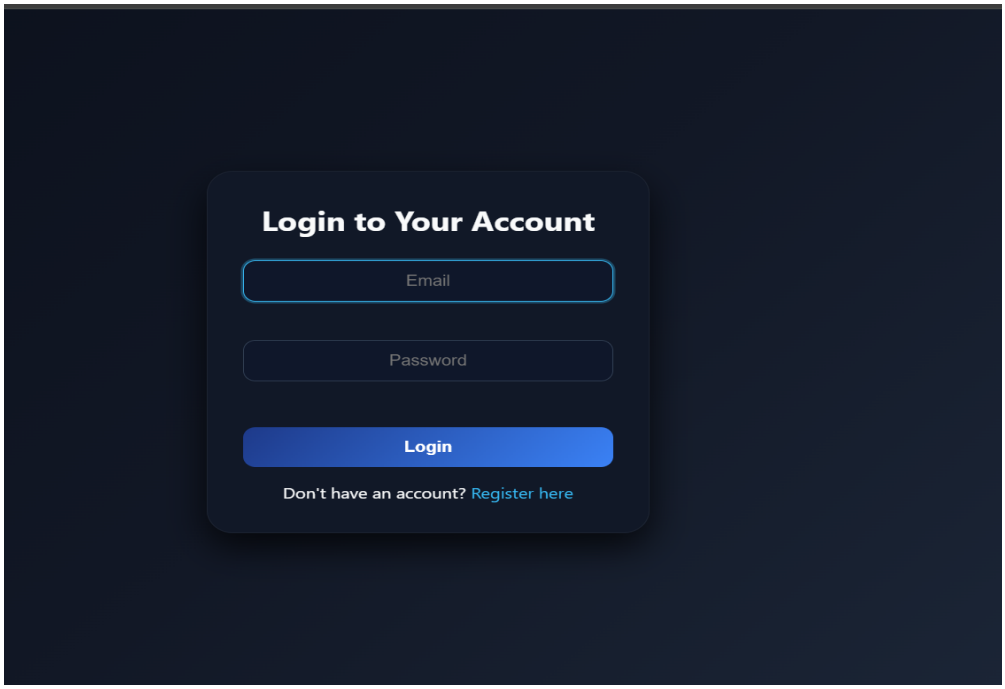


Figure 8.2: Login Page

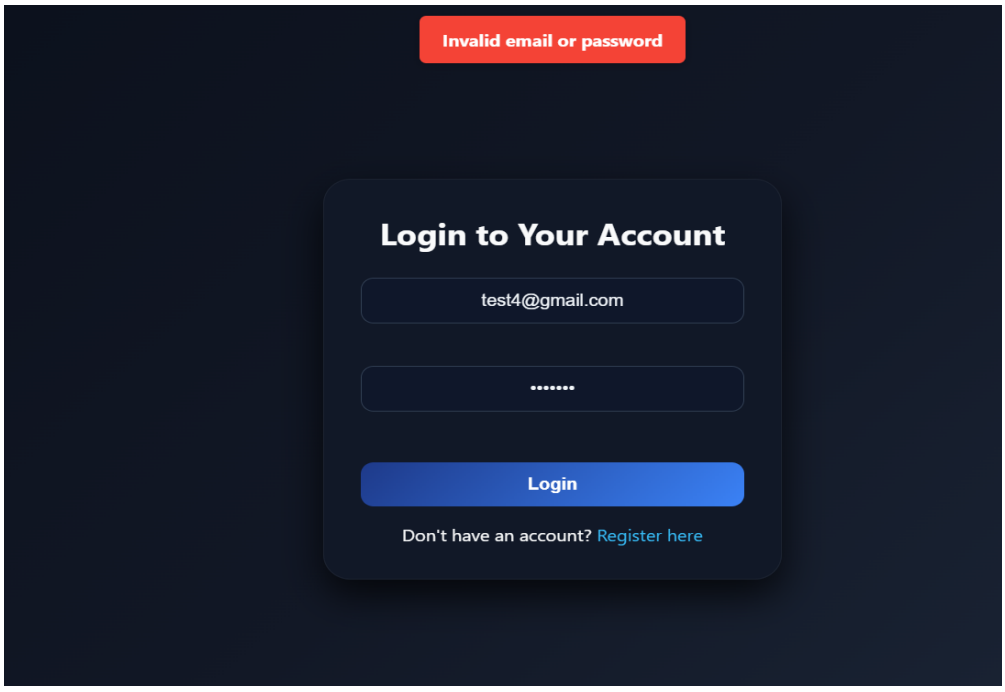


Figure 8.3: Login Page Error due to Invalid Email or Password

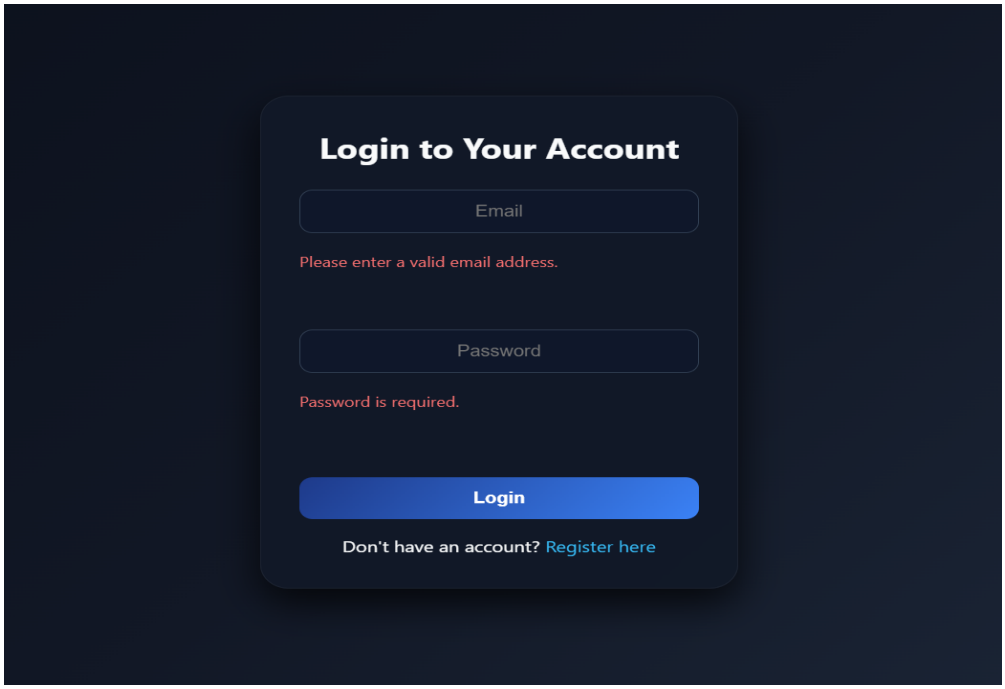


Figure 8.4: Login Page Error due to Empty Email-ID or Password

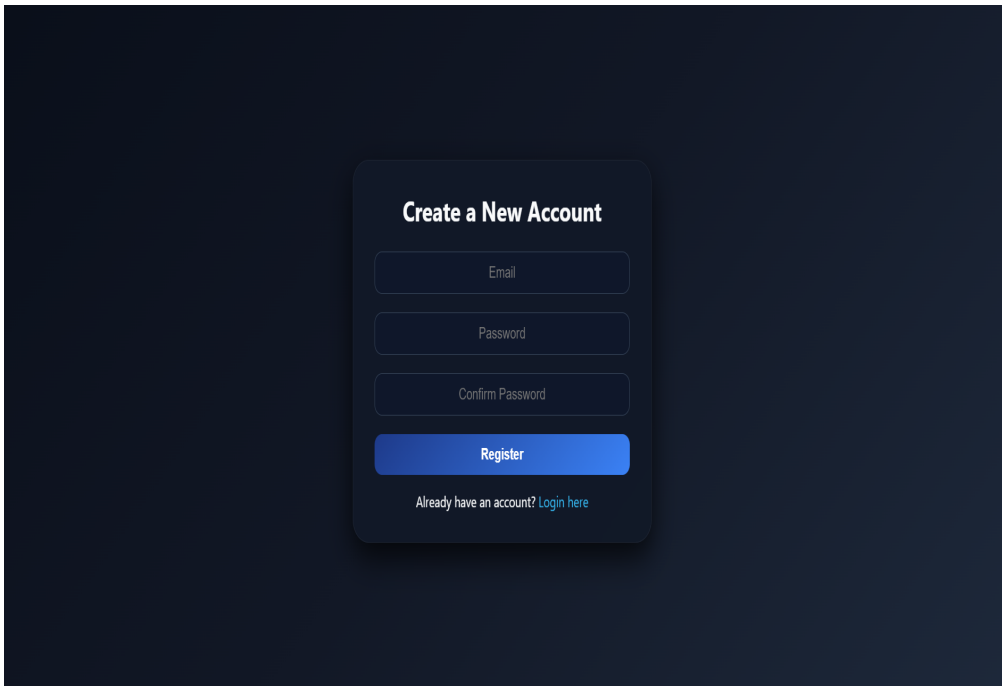


Figure 8.5: Registration Page

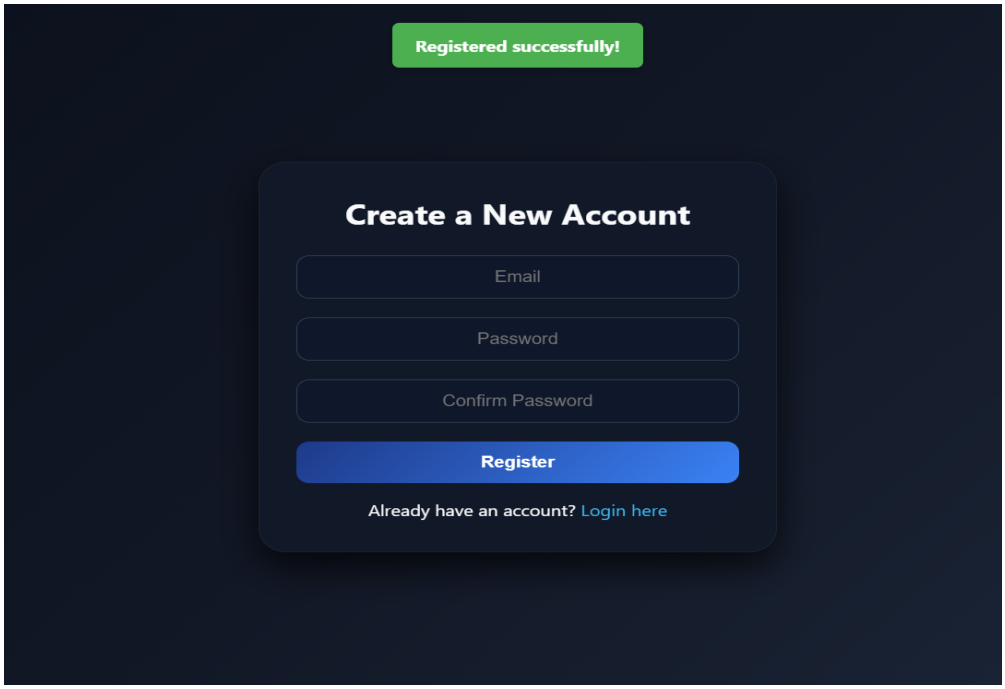


Figure 8.6: Successful Registration

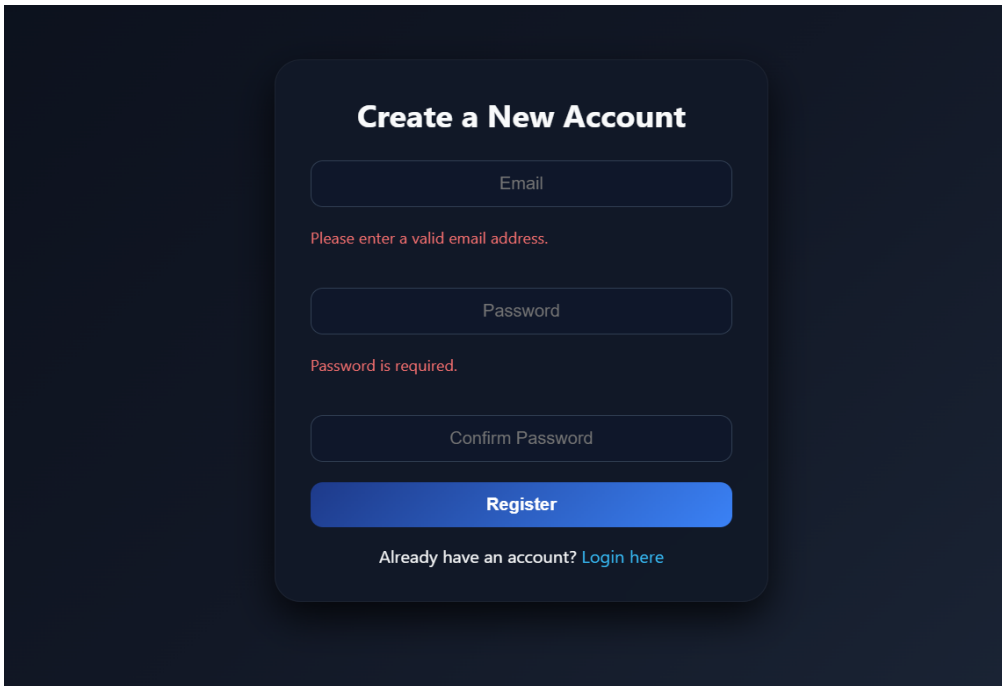


Figure 8.7: Invalid Registration

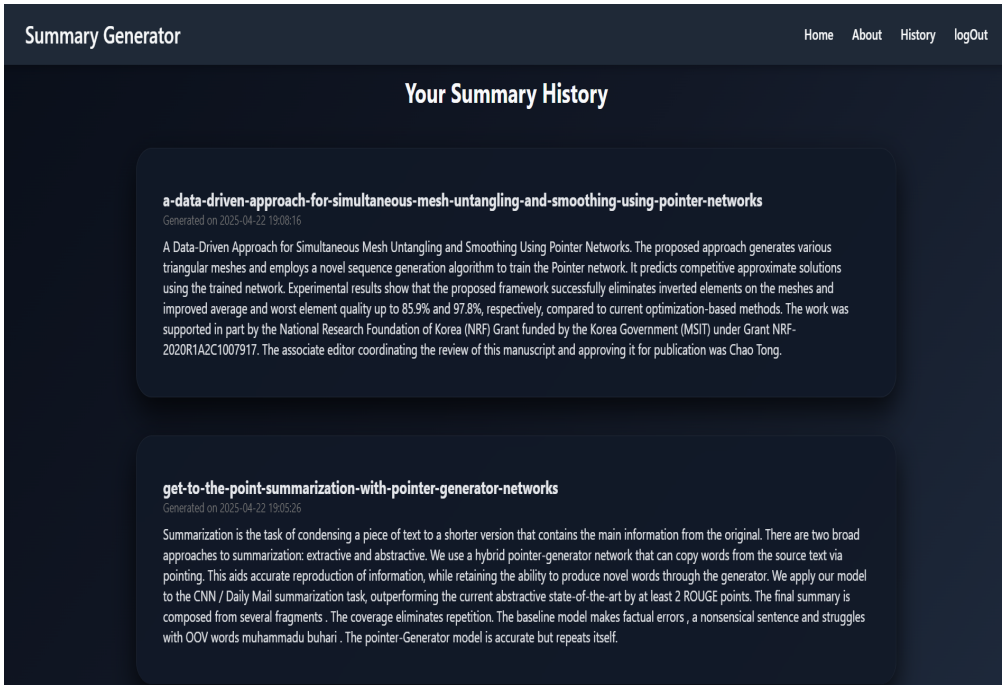


Figure 8.8: User's Summarization History Page

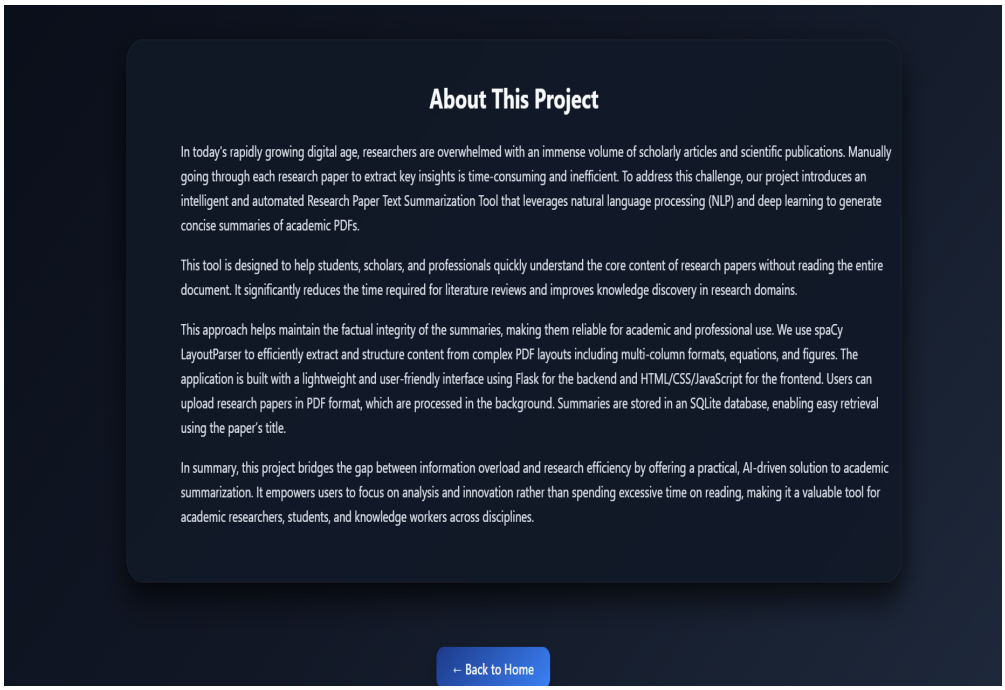


Figure 8.9: About Us Page

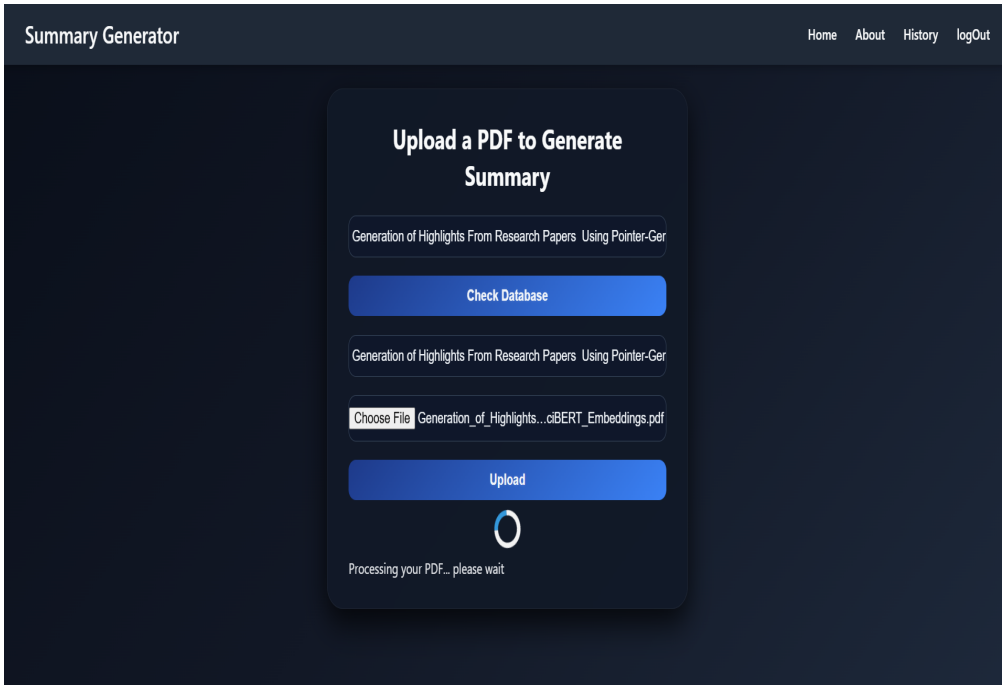


Figure 8.10: Output Processing Page

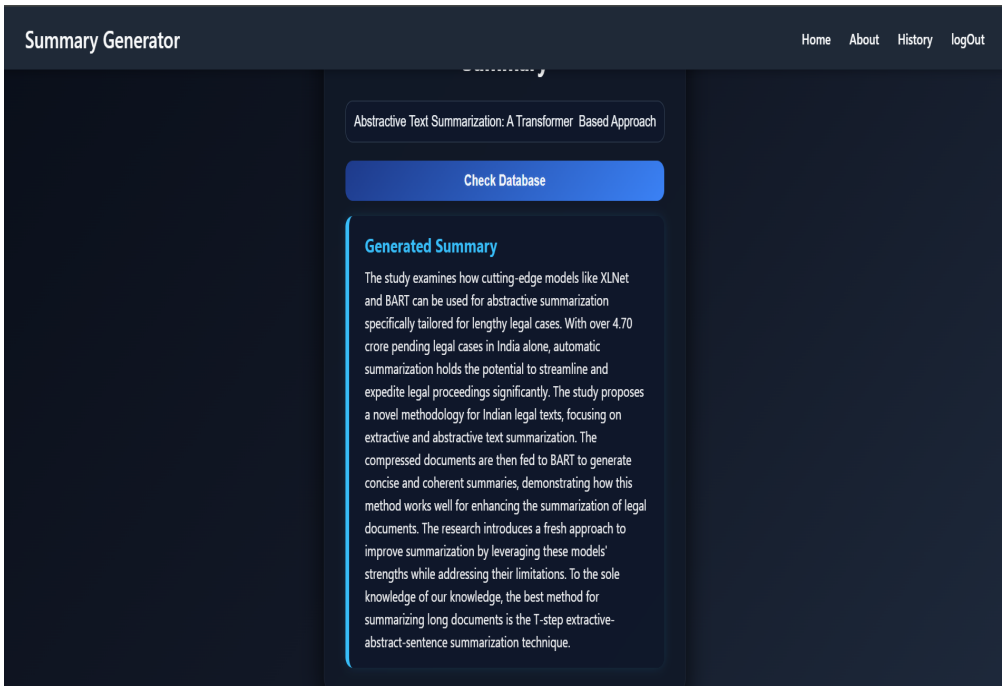


Figure 8.11: Successfully Generated Summary

CHAPTER 9
CONCLUSIONS

9.1 Conclusion

The project aimed to simplify the process of understanding complex research papers by leveraging modern NLP techniques for abstractive summarization. Through the integration of the BART model, known for its powerful sequence-to-sequence architecture, and the Pointer-Generator Network (PGN) for refining the generated outputs, we were able to develop a system that delivers concise and coherent summaries of academic content. The use of preprocessing techniques, effective model orchestration, and visualization tools further contributed to the usability and effectiveness of the solution. The modular architecture also allows for future expansion, including multi-lingual support and domain-specific fine-tuning.

In conclusion, the system provides a practical tool for researchers, students, and professionals by saving time and enhancing comprehension of scholarly documents. It not only demonstrates the potential of deep learning in real-world applications but also opens avenues for further research into adaptive summarization based on user preferences or reading levels. With iterative development and continuous integration of user feedback, this platform can evolve into a robust academic assistant capable of transforming how users engage with academic literature.

9.2 Future Work

- **Domain-Specific Summarization:** TWhile the current model is trained on general academic text, future iterations can incorporate domain-specific fine-tuning (e.g., medical, legal, or scientific domains). This will improve the contextual relevance and accuracy of summaries in niche fields where terminologies and writing styles differ significantly.
- **User-Customized Summaries:** Adding support for user preferences—such as summary length, technicality level, or emphasis on specific sections—can make the system more personalized and useful. This can be achieved by incorporating user feedback into the summarization pipeline or using reinforcement learning approaches.
- **Multi-Lingual Support:** Extending the system to handle non-English research papers or generate summaries in multiple languages would broaden its accessibility. Leveraging multilingual models such as mBART or using translation pipelines can help reach non-English speaking researchers and global users.
- **Integration with Research Databases:** Future versions can connect directly with platforms like arXiv, IEEE Xplore, or Google Scholar to fetch papers and generate summaries automatically. This would streamline the user workflow by allowing one-click summarization of recently published papers.
- **Real-time Summarization with Speech Input:** Adding functionality to summarize spoken content from research presentations or webinars us-

ing speech-to-text combined with summarization models would create new use cases. It can benefit students and professionals who attend lectures or conferences and wish to extract concise takeaways quickly.

9.3 Applications

- **Academic Research Portals and Digital Libraries:**Platforms like IEEE Xplore, arXiv, and SpringerLink can integrate this summarization system to provide quick, human-like summaries of long research articles. This helps users get the gist of a paper without reading through complex abstracts or full documents, improving research efficiency and literature scanning for scholars and students.
- **Research Paper Recommendation Systems:** When used in combination with AI-based recommender systems, the summaries generated by this project can be shown alongside recommended papers. This allows users to instantly evaluate whether a recommended article is relevant to their current study or project, without opening and reading the entire document.
- **Scientific News & Article Aggregators:** Websites or services that report on recent scientific breakthroughs can use the summarization system to convert complex academic papers into readable summaries for the general public. This bridges the gap between technical literature and lay audiences, aiding science communication and outreach.
- **Digital Libraries and Institutional Repositories:** Universities and research institutions often maintain digital libraries of theses, dissertations, and publications. Embedding this summarization system into such portals can help students and faculty quickly interpret archived research, enhancing access to knowledge and promoting reusability of academic content.
- **Academic Writing Assistants and Tools:** Mobile applications targeted at students (like Notion, Mendeley, or specialized study aids) can benefit from incorporating this feature to provide on-the-go summarization of PDFs, helping students to study smarter and manage their reading loads more effectively.
- **Corporate R&D Knowledge Management:** Companies with R&D departments often need to stay updated with the latest scientific advancements. Integrating this summarization tool can help professionals digest technical publications, patents, or whitepapers more efficiently, supporting faster innovation and informed decision-making.

CHAPTER 10
APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

The proposed system aims to develop a real-time abstractive summarization tool for research and educational purposes. To assess its feasibility:

- **Satisfiability:** The problem is satisfiable as the summarization of academic texts using transformer-based models and hybrid extractive-abstractive approaches is a well-researched and implementable problem. Using current technologies such as Python, PyTorch, Hugging Face Transformers, Flask/Streamlit, and MongoDB, the entire system can be developed and deployed efficiently.
- **Complexity Classification:**
  - The system includes natural language text processing, sequence-to-sequence model inference, and evaluation metric computation.
  - These tasks are computationally tractable and fall under **P-Type** complexity, as they involve polynomial-time operations such as tokenization, attention-based encoding/decoding, and string matching (e.g., ROUGE score).
- **Mathematical Representation:**
  The system can be modeled mathematically as:

  $$S = \{I, P, O, F, D, A, R\}$$

  where:
  - I = Inputs (e.g., research paper text, user queries)
  - P = Processes (e.g., preprocessing, model inference)
  - O = Outputs (e.g., abstractive summaries)
  - F = Functions (e.g., text extraction, BART/PGN application)
  - D = Database (e.g., MongoDB for summary storage)
  - A = Actors (e.g., User, System, Admin)
  - R = Rules (e.g., retry on failure, summary length threshold, token limit)
- **Modern Algebra Feasibility:**
  The flow of the system can be visualized as function mappings where:

  $$f : I \to O$$

  meaning that the system maps each valid input (research paper text) to a unique output (summary) through a defined sequence of transformation rules. This satisfies the criteria for a functionally complete and logically implementable system, which could also be modeled using Finite State Machines (FSM) for transition visualization between stages like login, processing, and output generation.

Hence, the proposed problem is logically consistent, solvable using polynomial-time methods, and does not fall under NP-Complete or NP-Hard computational classes.

10.2 Appendix B: Paper Publication Details

The research and development work from this project has culminated in a paper publication, successfully accepted and published in an international journal.

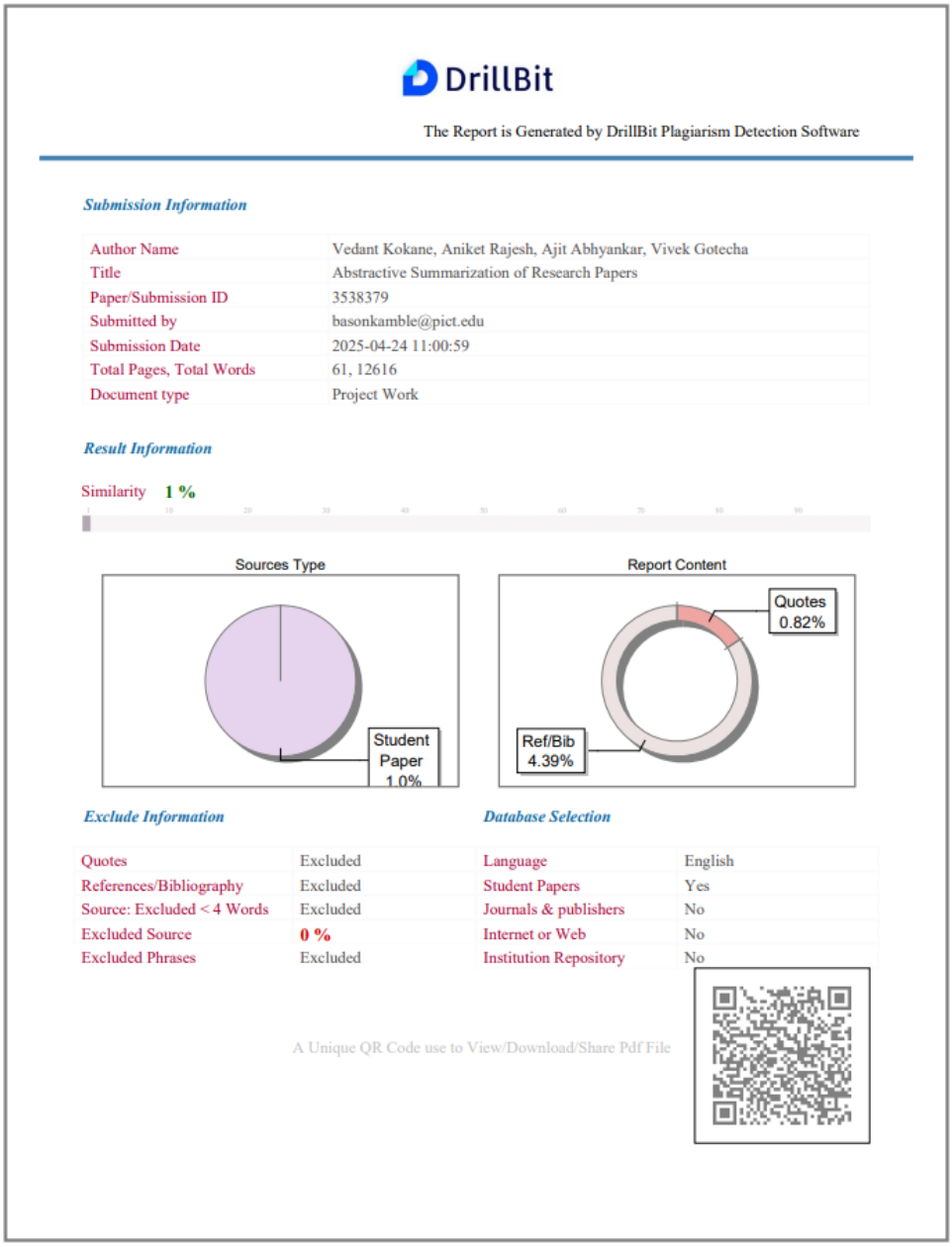
- **Paper Title:** Abstractive Summarization of Research Papers using BART Model
- **Paper ID:** VZIP-3116
- **Name of Journal:** VDI-Z Integrierte Produktion Journal
- **Volume and Issue:** Volume 12 Issue IV, April 2025
- **Status:** Paper Published and Accessible Online.
- **Certificates:**

Figure 10.1: Certificates of Publication

All documentation related to the publication, including screenshots of submission, certificates, and acceptance letters are provided in the annexed pages.

10.3 Appendix C: Plagiarism Report

Tool Used: https://www.drillbitplagiarism.com/



DrillBit

The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Vedant Kokane, Aniket Rajesh, Ajit Abhyankar, Vivek Gotecha
Title	Abstractive Summarization of Research Papers
Paper/Submission ID	3538379
Submitted by	basonkamble@pict.edu
Submission Date	2025-04-24 11:00:59
Total Pages, Total Words	61, 12616
Document type	Project Work

Result Information

Similarity 1 %

Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 4 Words	Excluded
Excluded Source	0 %
Excluded Phrases	Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	No
Internet or Web	No
Institution Repository	No

A Unique QR Code use to View/Download/Share Pdf File

Figure 10.2: Plagiarism Report Snapshot for entire report

Thus, the overall plagiarism percentage is **1%.**

CHAPTER 11
REFERENCES

[1] Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get To The Point: Summarization with Pointer-Generator Networks. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1073–1083, Vancouver, Canada. Association for Computational Linguistics.

[2] Nie, Wenbo & Zhang, Wei & Li, Xinle & Yu, Yao. (2019). An Abstractive Summarizer Based on Improved Pointer-Generator Network. 515-520. 10.1109/YAC.2019.8787620.

[3] S. Ren and Z. Zhang, "Pointer-Generator Abstractive Text Summarization Model with Part of Speech Features," 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 2019, pp. 1-4, doi: 10.1109/ICSESS47205.2019.9040715.

[4] S. Mukherjee, A. Jatowt, R. Kumar, A. Jangra and S. Saha, "Can Multimodal Pointer Generator Transformers Produce Topically Relevant Summaries?," 2023 International Joint Conference on Neural Networks (IJCNN), Gold Coast, Australia, 2023, pp. 1-8, doi: 10.1109/IJCNN54540.2023.10192022.

[5] Z. Liu, A. Ng, S. Lee, A. T. Aw and N. F. Chen, "Topic-Aware Pointer-Generator Networks for Summarizing Spoken Conversations," 2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU), Singapore, 2019, pp. 814-821, doi: 10.1109/ASRU46091.2019.9003764.

[6] M. Ihori, N. Makishima, T. Tanaka, A. Takashima, S. Orihashi and R. Masumura, "MAPGN: Masked Pointer-Generator Network for Sequence-to-Sequence Pre-Training," ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Toronto, ON, Canada, 2021, pp. 7563-7567, doi: 10.1109/ICASSP39728.2021.9414738.

[7] M. Liu, Z. Mu, J. Sun and C. Wang, "Data-to-text Generation with Pointer-Generator Networks," 2020 IEEE International Conference on Advances in Electrical Engineering and Computer Applications(AEECA), Dalian, China, 2020, pp. 244-251, doi: 10.1109/AEECA49918.2020.9213600.

[8] Z. Lin, Q. Zhou and L. Cao, "Two-Stage Encoder for Pointer-Generator Network with Pretrained Embeddings," 2021 16th International Conference on Computer Science & Education (ICCSE), Lancaster, United Kingdom, 2021, pp. 524-529, doi: 10.1109/ICCSE51940.2021.9569678.

[9] T. Rehman, D. K. Sanyal, S. Chattopadhyay, P. K. Bhowmick and P. P. Das, "Generation of Highlights From Research Papers Using Pointer-Generator Networks and SciBERT Embeddings," in IEEE Access, vol. 11, pp. 91358-91374, 2023, doi: 10.1109/ACCESS.2023.3292300.

[10] Y. -C. Tsai and F. -C. Lin, "Paraphrase Generation Model Integrating Transformer Architecture, Part-of-Speech Features, and Pointer Generator Network," in IEEE Access, vol. 11, pp. 30109-30117, 2023, doi: 10.1109/ACCESS.2023.3260849

[11] DZ. Li, Z. Peng, S. Tang, C. Zhang and H. Ma, "Text Summarization Method Based on Double Attention Pointer Network," in IEEE Access, vol. 8, pp. 11279-11288, 2020, doi: 10.1109/ACCESS.2020.2965575.

[12] A. M. Ahmed Zeyad and A. Biradar, "Advancements in the Efficacy of Flan-T5 for Abstractive Text Summarization: A Multi-Dataset Evaluation Using ROUGE and BERTScore," 2024 International Conference on Advancements in Power, Communication and Intelligent Systems (APCI), KANNUR, India, 2024, pp. 1-5, doi:10.1109/APCI61480.2024.10616418

[13] S. Modi and R. Oza, "Review on Abstractive Text Summarization Techniques (ATST) for single and multi documents," 2018 International Conference on Computing, Power and Communication Technologies (GUCON), Greater Noida, India, 2018, pp. 1173-1176, doi: 10.1109/GUCON.2018.8674894.

[14] S. Rafi and R. Das, "Abstractive Text Summarization Using Multimodal Information," 2023 10th International Conference on Soft Computing & Machine Intelligence (ISCMI), Mexico City, Mexico, 2023, pp. 141-145, doi: 10.1109/ISCMI59957.2023.10458505

[15] M. F. Mridha, A. A. Lima, K. Nur, S. C. Das, M. Hasan and M. M. Kabir, "A Survey of Automatic Text Summarization: Progress, Process and Challenges," in IEEE Access, vol. 9, pp. 156043-156070, 2021, doi:10.1109/ACCESS.2021.3129786

LIST OF ABBREVIATIONS

Abbreviation	Illustration
BART	Bidirectional and Auto-Regressive Transformers
BERT	Bidirectional Encoder Representations from Transformers
PGN	Pointer Generator Network
OOV	Out of Vocabulary
ROUGE	Recall Oriented Understudy of Gisting Evaluations
POS	Part of Speech
Seq2Seq	Sequence to Sequence
GloVe	Global Vectors for Word Representation
MIT	Multimodality Image Text
LSTM	Long Short Term Memory
GloVe	Global Vectors for Word Representation
BLEU	BiLingual Evaluation Understudy

LIST OF FIGURES

LIST OF TABLES